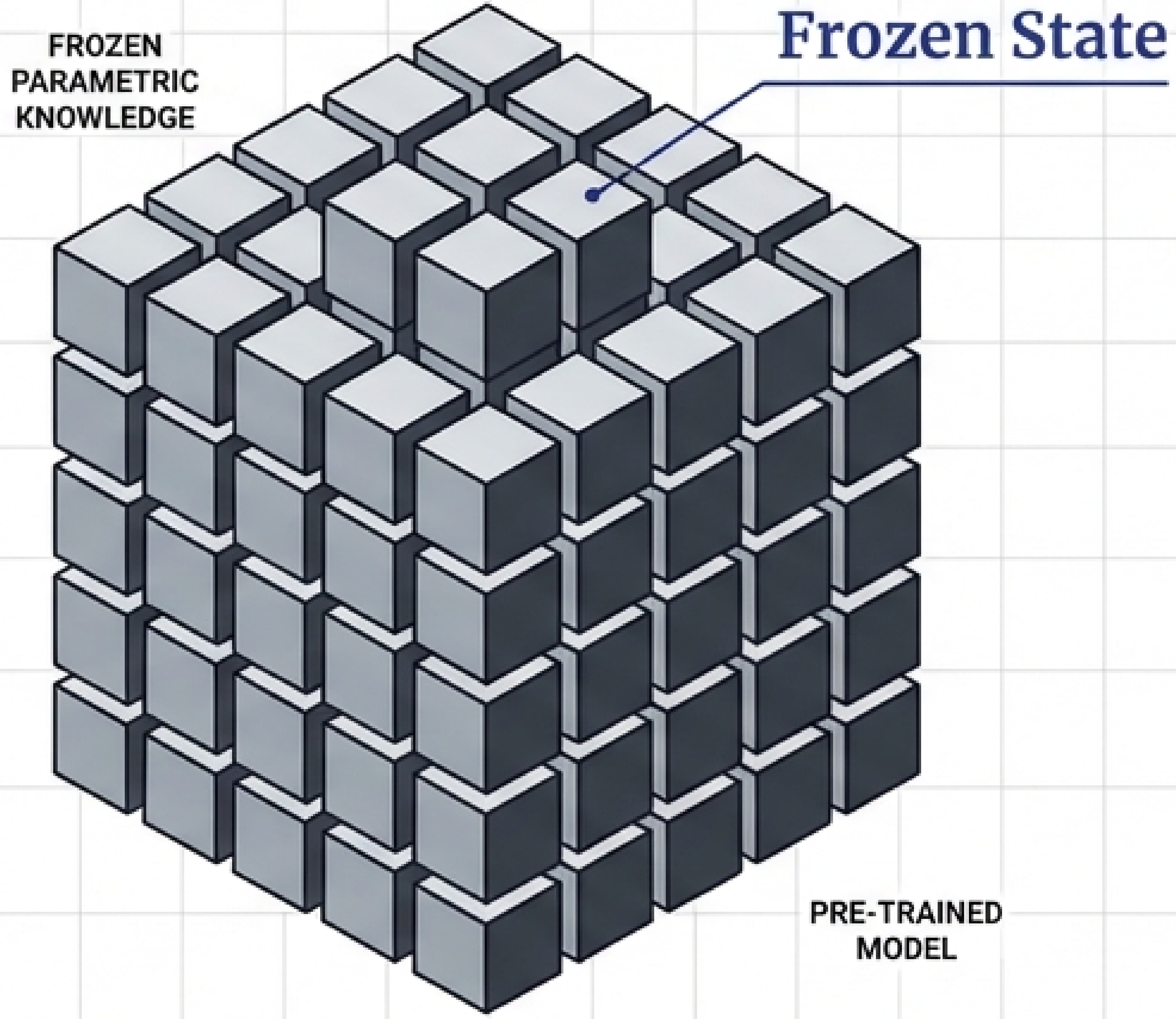
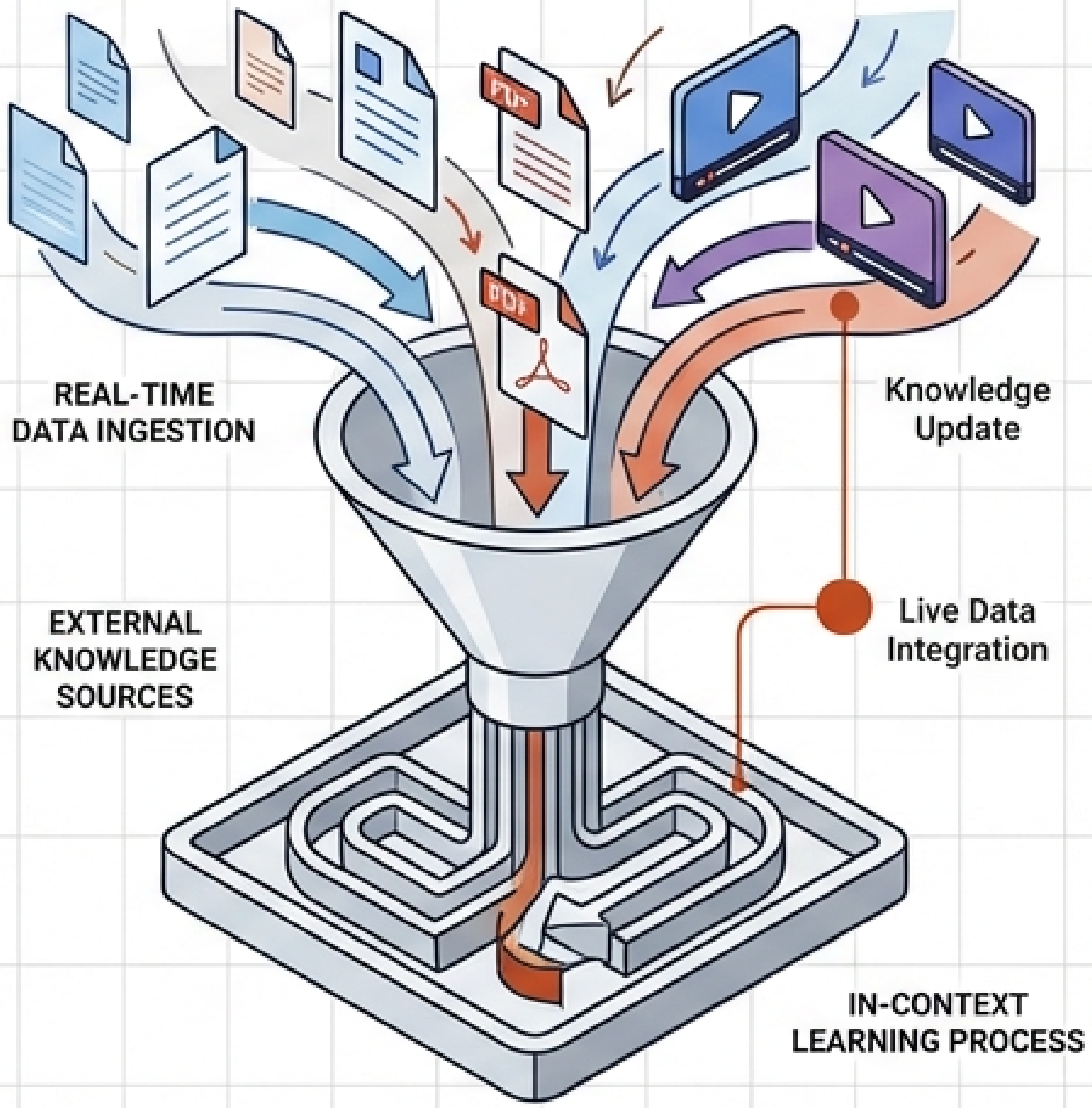
Retrieval Augmented Generation

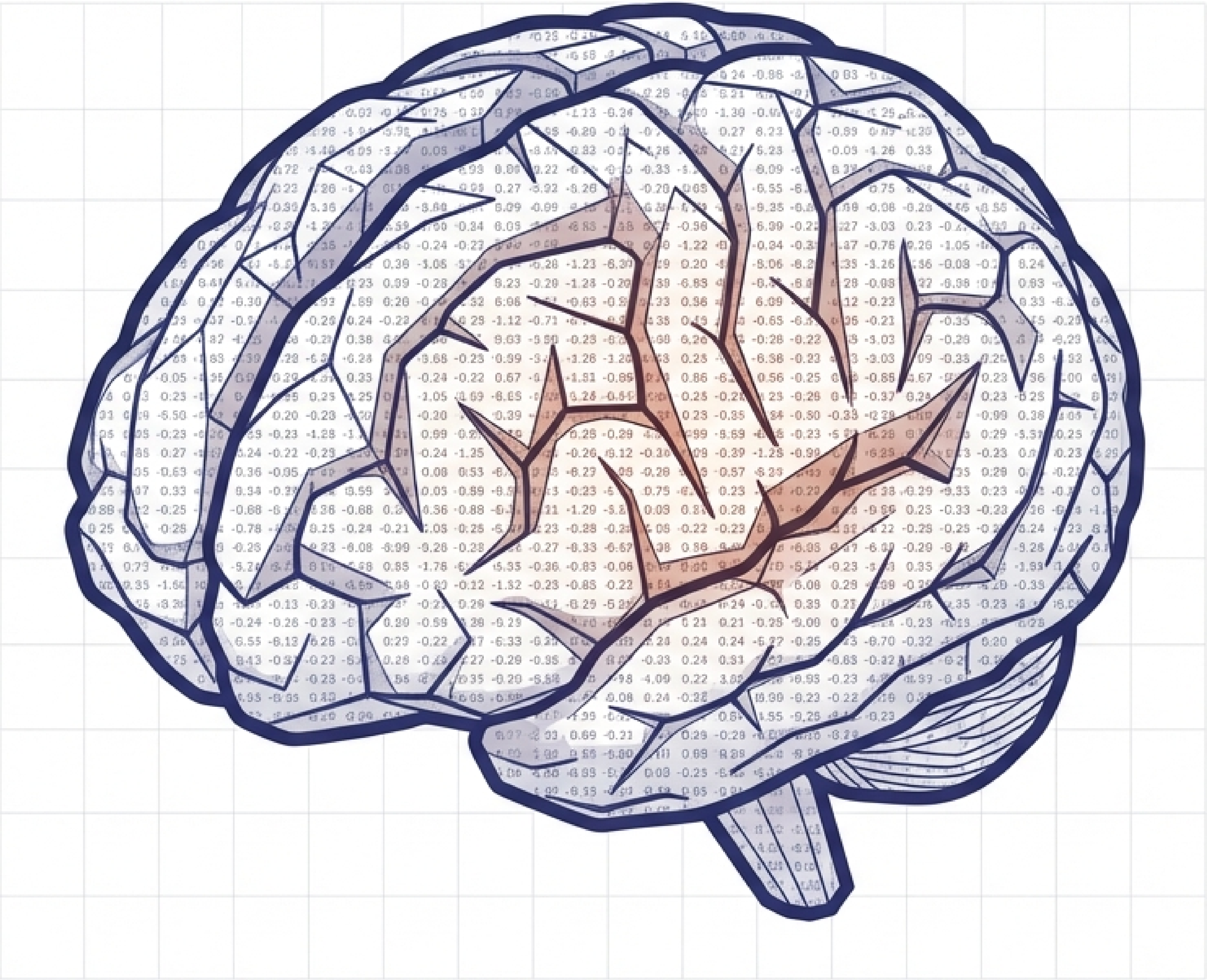
Bridging the Knowledge Gap: From Parametric Knowledge to In-Context Learning





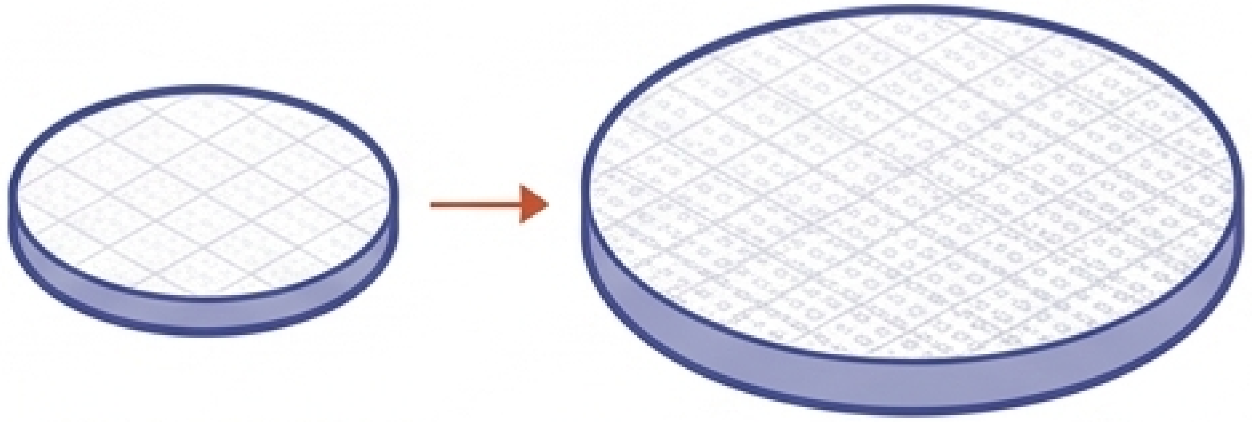
A Strategic Deep-Dive based on CampusX / Nitish

LLMs store the world's knowledge in frozen parameters



Parametric Knowledge:

Models like GPT-4 do not access a live database. They store patterns in parameters (weights and biases) acquired during pre-training.

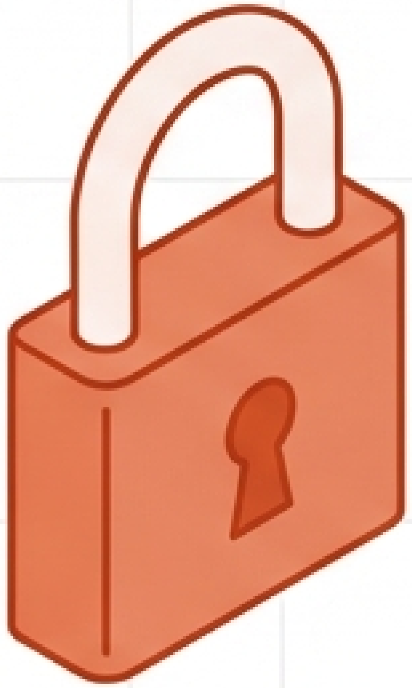


7B Parameters
(Smaller Capacity)

70B Parameters
(Larger Capacity)

Scale = Capacity. More parameters allow for more stored knowledge, but it remains a static snapshot of the internet at the time of training.

The ‘Smart’ Model fails in three critical scenarios



1. Private Data

LLMs cannot access internal documents, company wikis, or course videos. They only know public internet data seen during training.

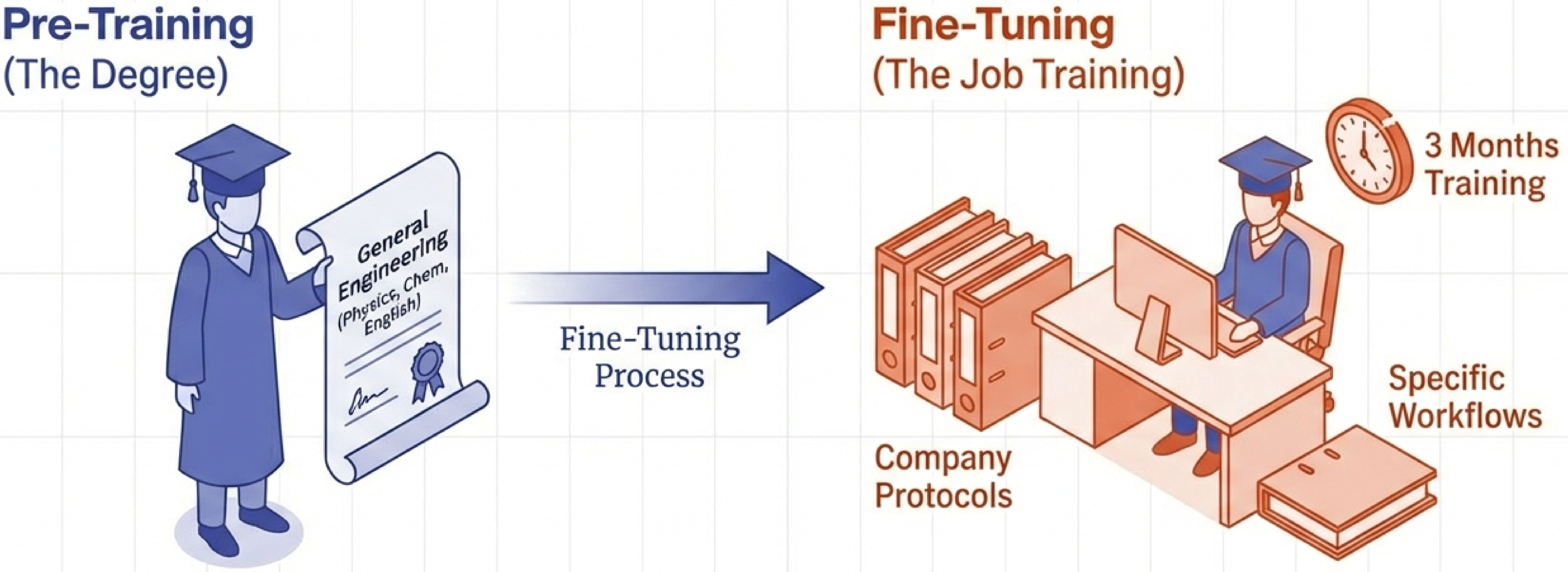
2. Recent Data (Knowledge Cut-off)

Models are frozen in the past. If training stopped in January, they cannot answer “What happened in the news today?”

3. Hallucinations

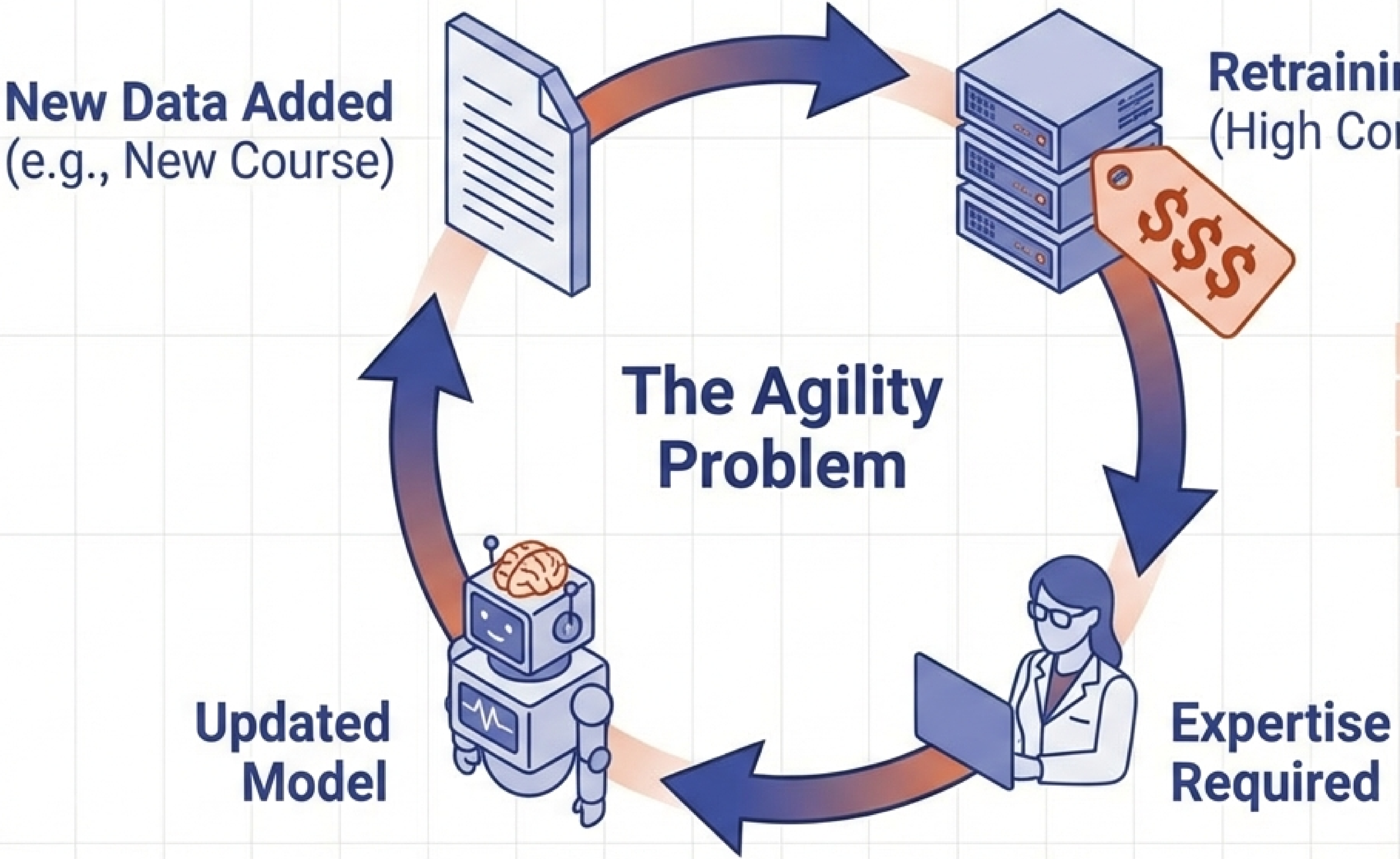
Models are probabilistic, not factual. They often confidently invent facts when they don’t know the answer (e.g., claiming Einstein played professional football).

The first attempt at a solution: Fine-Tuning



Fine-tuning is like job training. We take a generalist model and re-train it on a smaller, domain-specific dataset to 'bake' new knowledge into its parameters.

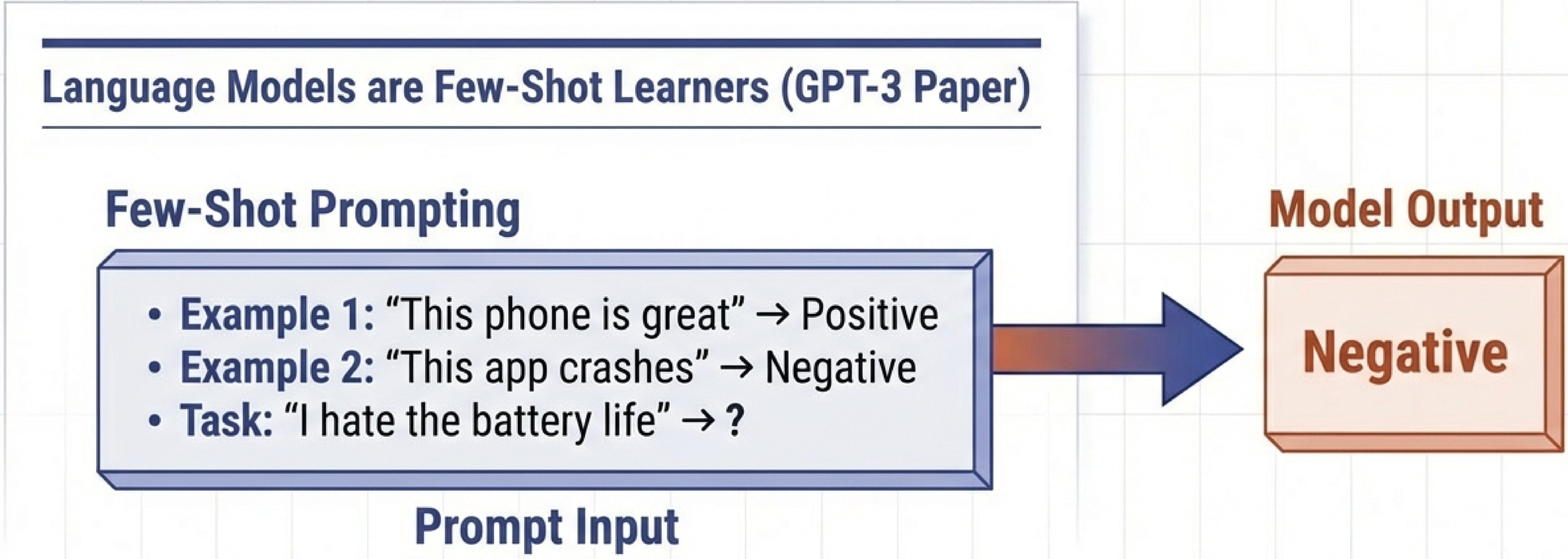
Why Fine-Tuning is often too heavy, costly, and static



If data changes daily (dynamic data), you must retrain continuously. The model is obsolete the moment a new document is created.

The Paradigm Shift: In-Context Learning (ICL)

The emergent property that changed everything.



Researchers discovered that large models (175B+) can learn a task simply by seeing examples in the prompt, without any weight updates.

RAG injects the 'Textbook' directly into the exam

User Query

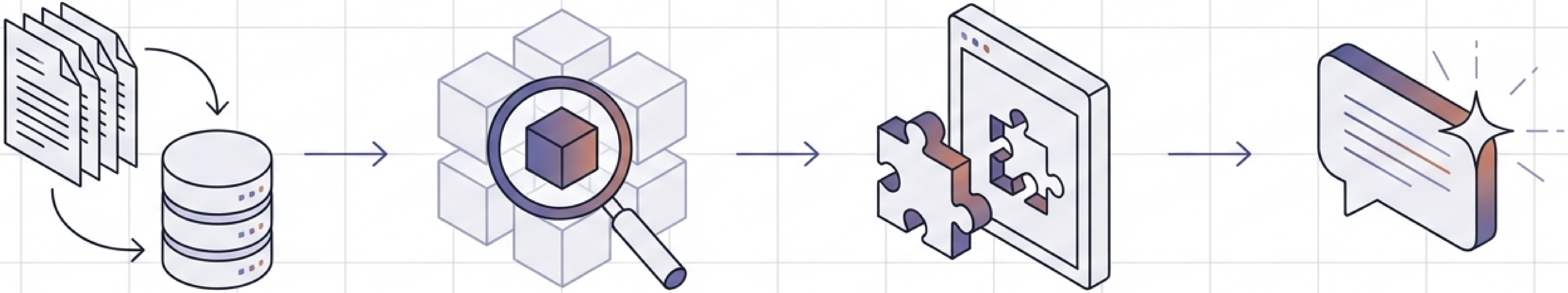
External Context (Retrieved Facts)

Intelligent, Grounded Answer

Retrieval Augmented Generation (RAG) evolves In-Context Learning. Instead of providing examples of logic, we retrieve the exact information needed (e.g., a specific lecture transcript on **Gradient Descent**) and feed it into the **prompt**. It shifts the paradigm from **Memorisation** to **Open Book**.

The Architecture: Merging Information Retrieval with Text Generation

1. Indexing | **2. Retrieval** | **3. Augmentation** | **4. Generation**



Preparing the Knowledge Base

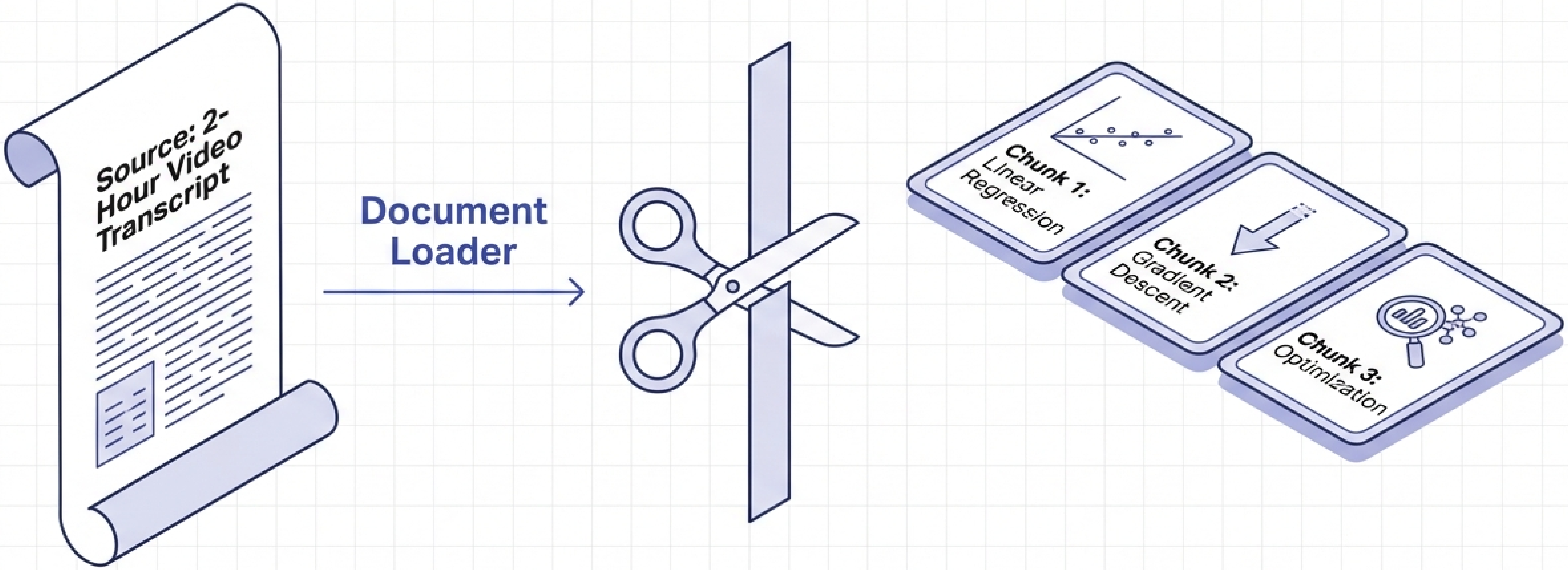
Finding Relevant Data

Constructing the Contextual Prompt

Producing the Answer

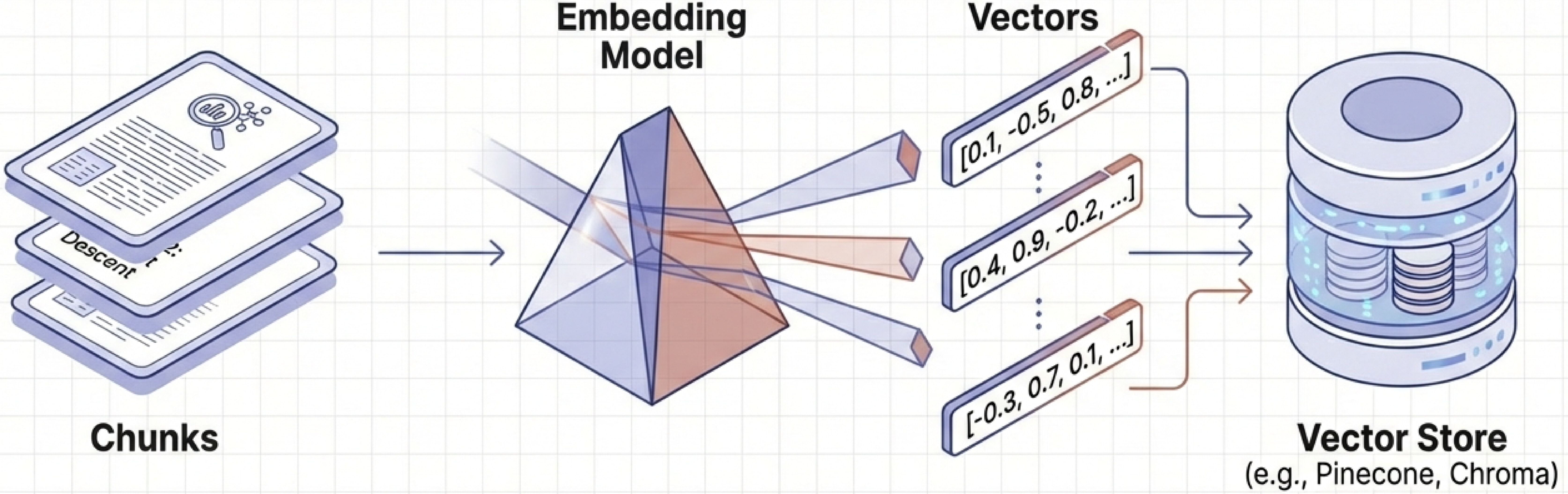
RAG combines the precision of Computer Science search with the creativity of Generative AI.

Step 1: Indexing – Ingestion and Chunking



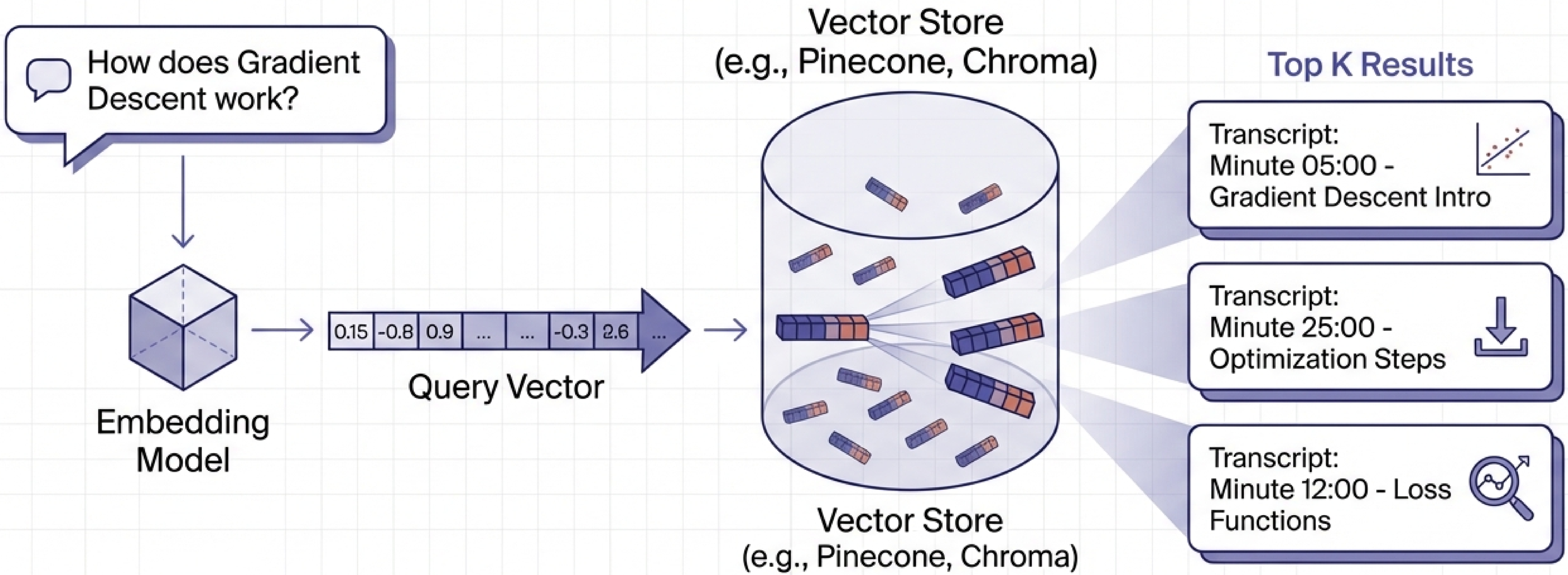
Ingestion loads data from PDFs or Web. Chunking splits large texts into semantically meaningful pieces to fit the LLM's context window and improve search precision.

Step 1: Indexing – Embeddings and Vector Stores



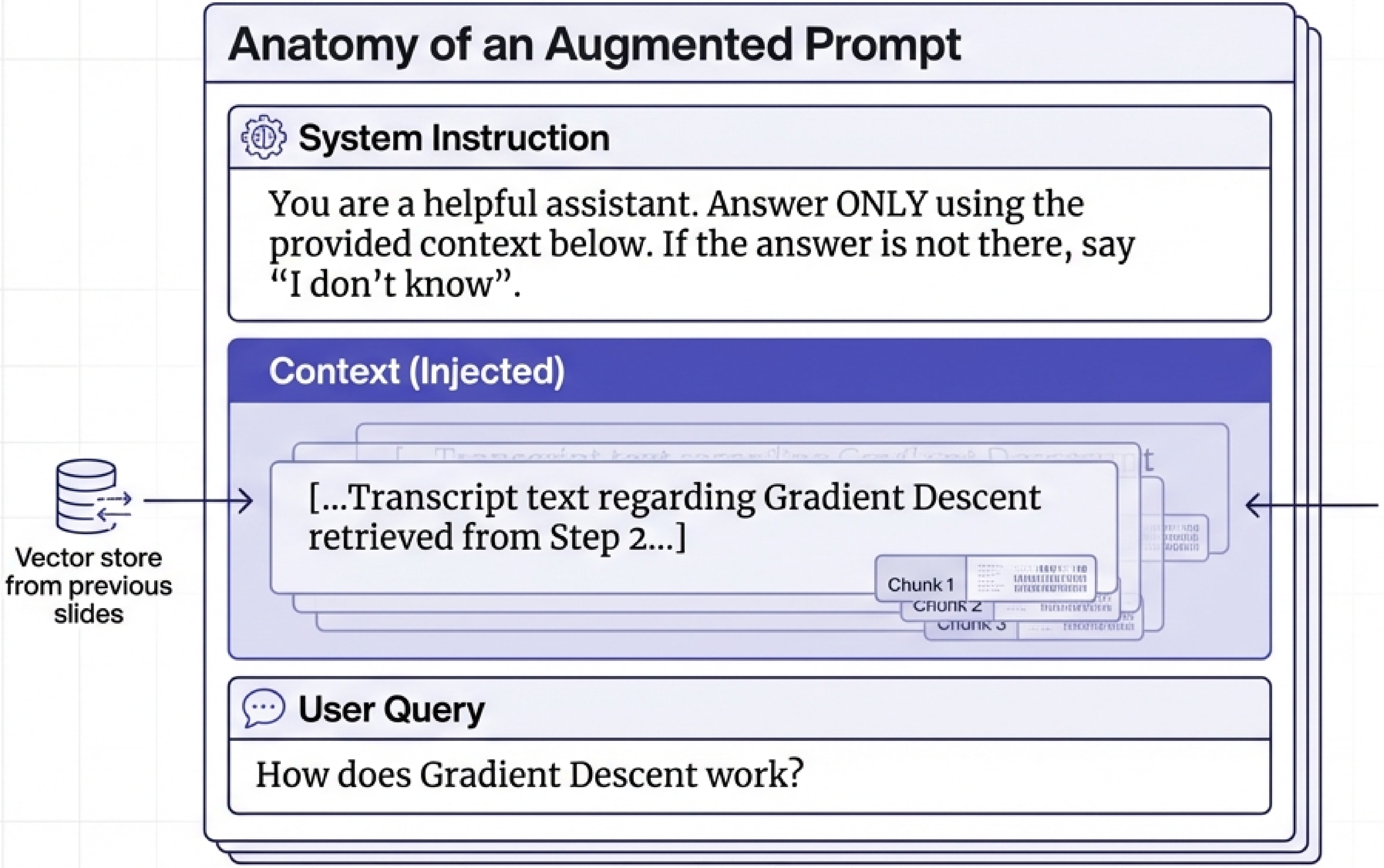
Embeddings convert text into dense vectors—mathematical representations of semantic meaning. These vectors are stored in a Vector Database for high-speed similarity searching.

Step 2: Retrieval – Semantic Search in Real-Time



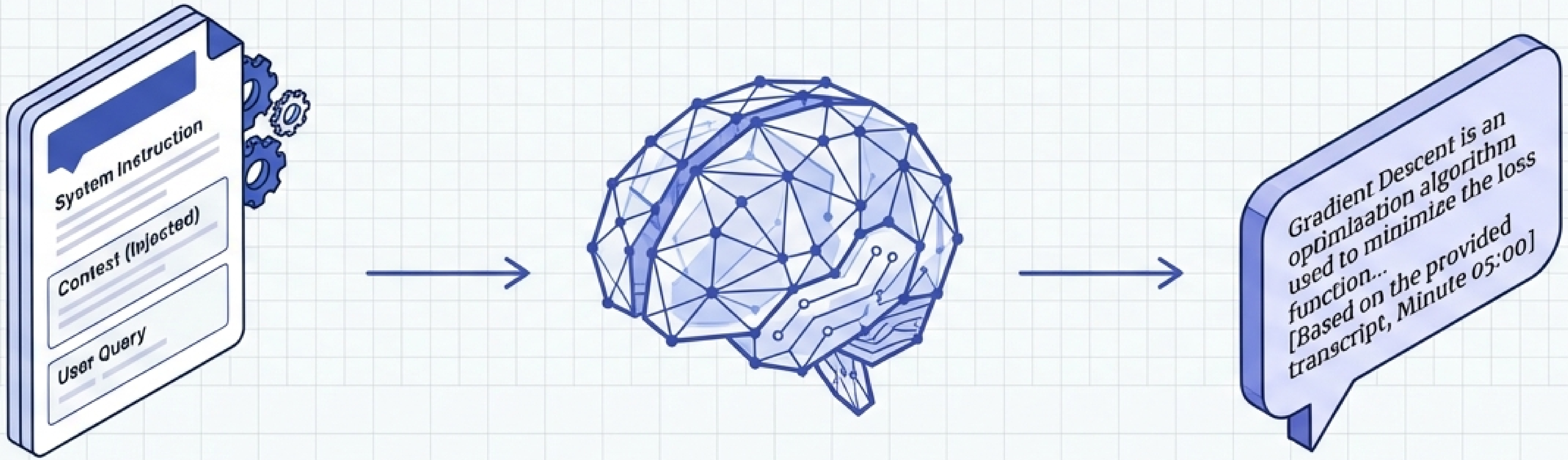
Semantic Search calculates mathematical proximity (Cosine Similarity) to retrieve only the most relevant chunks, not the whole document.

Step 3: Augmentation – Engineering the Prompt



We construct a **composite prompt that constraints the model.** By injecting the retrieved chunks as context, we force the LLM to ground its answer in our specific data.

Step 4: Generation – Grounded Responses



Augmented Prompt
(from previous slide)

LLM

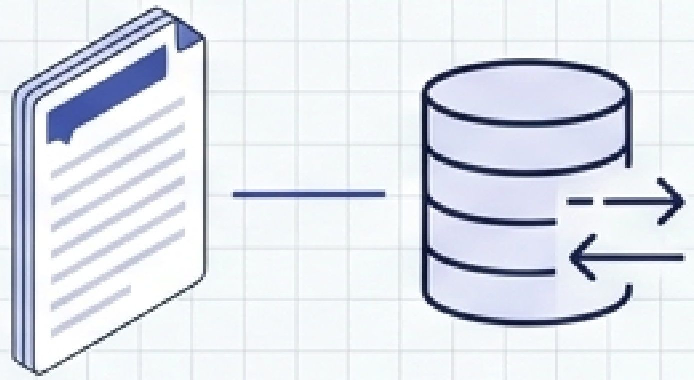
Chat Response

The LLM uses its linguistic capabilities to parse the context and formulate a coherent answer. Because it is instructed to use only the provided context, hallucinations are minimized and the answer is specific to your private data.

Closing the Loop: How RAG solves the three flaws

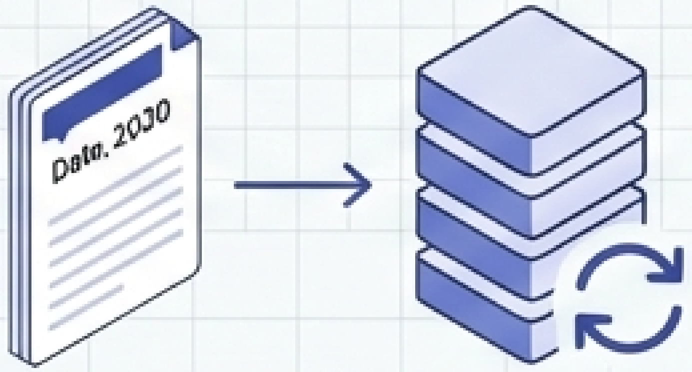
Private Data **SOLVED.**

The Vector Store acts as the external memory for your specific documents.



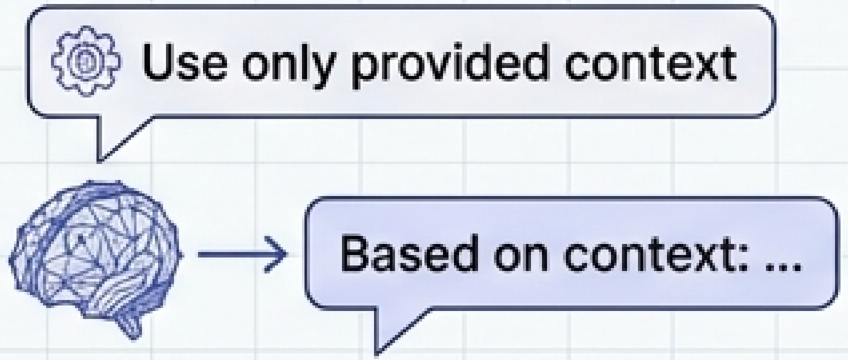
Recent Data **SOLVED.**

Agile updates. Just add new docs (e.g., today's news) to the store—no retraining needed.

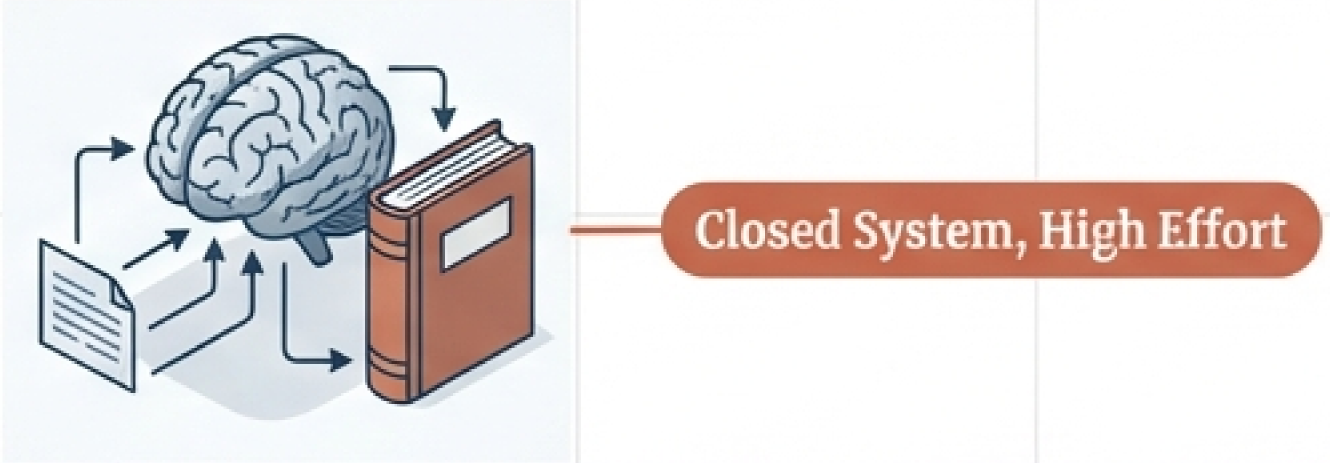


Hallucinations **SOLVED.**

Grounding. Instructions to "use only provided context" prevent fabrication.



The Verdict: RAG vs. Fine-Tuning

Fine-Tuning	RAG (Retrieval Augmented Generation)
Analogy: "Memorising the Textbook"	**Analogy:** "Open Book Exam"
	
• **Best For:** Adapting Tone, Style, or specialized vocabulary. • **Cost:** High (Compute intensive). • **Maintenance:** Difficult (Requires retraining).	• **Best For:** Accuracy, Dynamic/Recent Data, Factual Grounding. • **Cost:** Low (One-time setup, cheap storage). • **Maintenance:** Easy (Just add/remove documents).

For most knowledge-retrieval applications, RAG is the agile, cost-effective standard.